



JPEG Coding

Uğur ALTINIŞIK – Oğuz ÖZÜM

181201033 - 221211043

ELE 474 – Dijital Sinyal İşleme | 18.12.2023

Giriş	2
Soru-1	3
Görüntü Ön İşleme, Discrete Cosine Transform Matrisi ve Quantization Tablosu.	3
Blok İşlemleri, DCT ve Quantization İşlemi.....	4
Matris Dönüşüm (ZigZag ve Inverse Zigzag) Algoritmaları.....	4
Huffman Kodlaması ve Kod Çözümü.....	6
Inverse DCT ve Dequantization İşlemi.....	7
Soru-2	8
Sonuç	11
Soru-1 Kodu	12
Soru-2 Kodu	17
Kaynakça	21

Giriş

Proje 2 parçadan oluşmaktadır. Projenin ilk kısmında JPEG encoder ve decoder tasarlanması beklenirken ikinci kısmında yazılmış olan JPEG encoder ve decoder değiştirilerek quantization aşaması adaptif şekilde gerçekleştirilecektir. Projenin bütün aşamalarında MATLAB kullanılmış olup ikinci kısımda desmos grafik çizici uygulaması da kullanılmıştır.

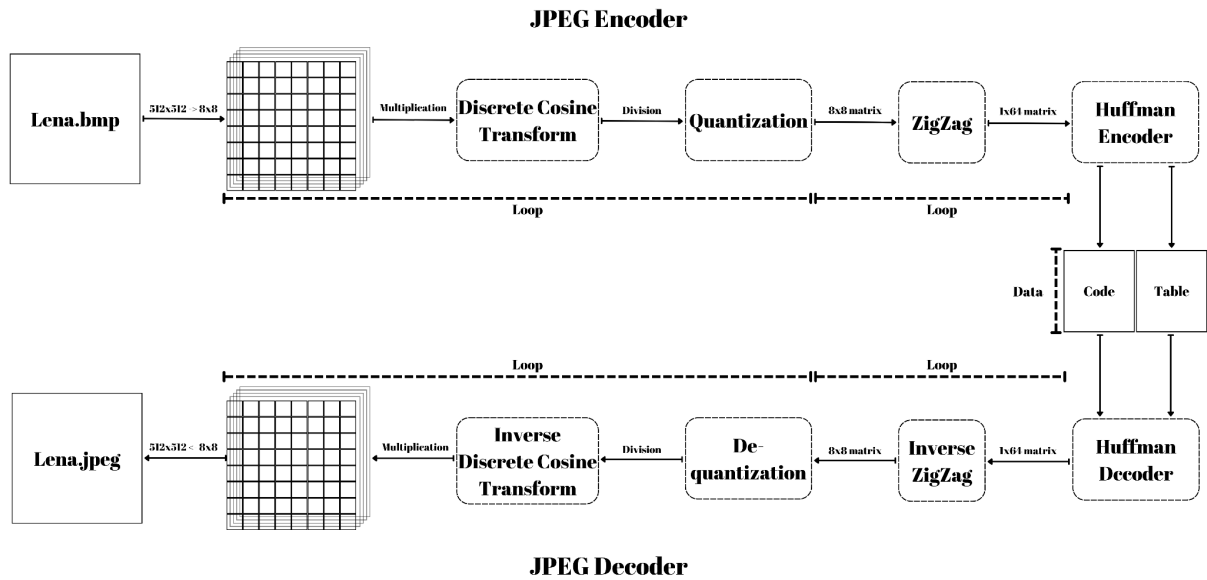
İlk olarak JPEG encoder algoritması yazılmıştır. Bu algoritmaya göre önceden verilmiş olan bitmap dosya türündeki Lena görseli matris formuna dönüştürülmüş ve bu matris 8x8 matris formlarına parçalanmıştır. Bu yeni 8x8 matrislere Discrete Cosine Transform uygulayarak frekans alanındaki değerleri elde edilmiş ve bu değerler de bir matrise kaydedilmiştir. Kaydedilen matrisler hazır olarak verilmiş Quantization matrisine bölünerek mevcut bit sayıları düşürülmesi hedeflenmiştir. Yine görselin boyutlarını düşürmek amacı ile bu matrislere Huffman Encoding uygulanmıştır. Huffman Encoder girdi olarak vektör almaktadır. Bu kapsamda 8x8 matrisleri 1x64 matrise dönüştürecek bir algoritma oluşturulmuştur. Bu algoritmayı oluşturma ve uygulama aşamaları Proje başlığı altında detaylı bir biçimde açıklanmıştır. Matrislere Huffman Encoding işlemi uygulaması çıktısı olan satır matrisi ile birlikte JPEG encoder aşaması tamamlanmıştır. Bir sonraki aşama olarak JPEG decoder algoritması oluşturulmuştur. Temel olarak bu algoritma, önceki aşamadaki bütün basamakların tersi olarak çalışmaktadır. Yani satır matrise Huffman Decoding işlemi uygulanmış, çıktı olarak daha çok bitle ifade edilen 1x64 matrisleri 8x8 matrise dönüştürecek algoritma oluşturulmuş, bu 8x8 matrisler Quantization matrisi ile çarpılarak görselin frekans alanındaki değerleri bulunmuş ve son aşama olarak Inverse Discrete Cosine Transform uygulanmıştır. Bu basamaklar sonucunda projenin ilk kısmı gerçekleştirilmiştir.

İkinci kısım için ise ilk soruda yazılan Quantization matrisi bir sabit ile çarpılarak çoklu Quantization matrisleri elde edilmesi hedeflenmiştir. Bu Quantization aşamasında kalitenin belli bir seviyenin altına düşmemesi ve bunun paralelinde maksimum sıkıştırma oranı elde edilmesi hedeflenmiştir. Bu kapsamda algoritma oluştururken resmin frekans değerlerindeki maksimum değerlerin bilgi içerdiği dolayısıyla bu verilerin korunması gereken veriler olduğu, minimum değerlerin içerdiği bilginin ise diğer duruma göre daha az önemli olduğu dolayısıyla bu verilerin daha az korunması gereken veriler olduğu göz önünde bulundurulmuştur. Bu kapsamda birden fazla metot ile bu maksimum ve minimum değerler test edilmiş, birden fazla metot ile bu Quantization matrisine çarpan olarak verilecek katsayılar hesaplanmıştır. Bütün bu testler ve katsayılar Proje bölümünde detaylı şekilde anlatılmıştır.

İlk aşama için kalite oranı ölçüm parametresi olan PSNR değeri 32'den yukarıda olması hedeflenmiş, ikinci aşama için ise yine PSNR değerinin minimum 30 olması ve maksimum sıkıştırma oranının elde edilmesi hedeflenmiştir.

Soru-1

JPEG encoder tasarlanması sürecinde sırasıyla bmp formatındaki dosya alınmış, matris formuna dönüştürülmüş, bu matrisler Discrete Cosine Transform ile frekans alanında bir matris elde edilmiş, bu matris Quantize edilerek matrise ZigZag uygulanmış, bu uygulanan ZigZag çıktısı Huffman Encoder'a giriş olarak verilerek sıkışmış formattaki görsel verileri ve bu verileri içeren veri yönerge tablosu elde edilmiştir. JPEG decoder tasarlanması sürecinde ise sırasıyla sıkışmış formattaki görsel verileri ve bu verileri içeren veri yönerge tablosu Huffman Decoder'a giriş olarak verilerek çıkış Inverse ZigZag fonksiyonuna verilmiş, Inverse ZigZag çıktısı Dequantize edilerek Inverse Discrete Cosine Transform'a tabi tutulmuştur. Elde edilen matris pixel değerlerine karşılık geliyor olup dosya formatı da jpeg biçiminde kayıt edilebilecektir. Bütün bu izlenen algoritma, fonksiyon girdi-çıktıları ve genel kod yapısı Şekil 1.1'de gösterilmiştir.



Şekil 1.1: JPEG Encoder ve Decoder Algoritması

Görüntü Ön İşleme, Discrete Cosine Transform Matrisi ve Quantization Tablosu

Projede ilk olarak 512x512 grayscale bir Bitmap görüntüsü olan "lena.bmp" dosyası MATLAB'ın "imread" fonksiyonu ile okunmuş, işleme yapılabilmesi için resim "double"a dönüştürülmüş ve görüntü piksel değerleri, DCT işlemine tabi tutulmadan önce -128 ve 127 değerlerinde sınırlandırılması için bu matristen 128 çıkarılmıştır. Bu işlemler sonucu görüntü işlenmeye hazır hale getirilmiştir.

Daha sonrasında DCT işleminde kullanılacak matris, sonraki bölümde detaylandırılacak olan fonksiyon ile oluşturulmuş ve DCT adından sonra kullanılacak olan quantization tablosu sabit bir şekilde kodlanmıştır.

Blok İşlemleri, DCT ve Quantization İşlemi

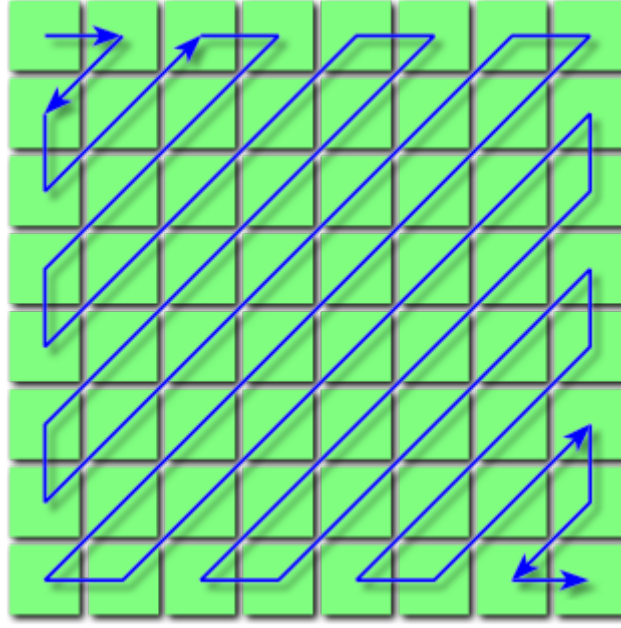
Projenin ilerleyen kısmında ön işleme yapılan resim 8x8'lik bloklara bölünmüş ve her bloğa Discrete Cosine Transform sonrasında da quantization işlemi uygulanmıştır.

DCT işlemi resmin sıkıştırılmasından önce gerçel frekans alanına geçirilebilmesi için yapılmaktadır. Burada bu işlem iki döngülü ve her pikseli teker teker işleyen algoritma kullanılarak değil, MATLAB ve benzeri yazılım ortamlarının matris işlemlerindeki etkinliği düşünülerek bir DCT matrisi oluşturularak yapılmıştır. Bu metodun avantajı her bloğun her pikseli üzerinde birden çok döngünün döndürülmesi yerine DCT matrisinin bir kere oluşturulması ve her bloğun bu matris ile sadece iki kez çarpılmasıdır (DCT matrisinin transpozunun görüntü bloğu ile çarpılması ve bu elde edilen matrisin DCT matrisi ile çarpılması). Bu iki çarpımdan biri satır, diğeri ise sütunların DCT'sini almaktadır bu şekilde tüm blok frekans alanında ifade edilmektedir. DCT matrisini oluşturan fonksiyon blok büyüklüğünü girdi olarak almaktadır ve DCT matrisini çıktı olarak vermektedir.

Quantization işlemi ise resmin bit dizisi olarak iletilmesi aşamasındaki ilk adım olarak JPEG algoritmasının önemli bir parçasıdır. Projenin bu aşamasında, DCT'den çıkan her bir görüntü bloğu quantization matrisine bölünmektedir ve her bloğun her değeri en yakın tam sayıya yuvarlanmaktadır. Bu işlem nicemlemenin yanı sıra kayıplı bir sıkıştırma sağlamaktadır ve görüntü sıkıştırmasının ilk adımı olarak kabul edilebilmektedir. Bu işlemler sonucu elde görüntünün her 8x8 bloğunun frekans alanında nicemlenmiş hali elde edilmektedir.

Matris Dönüşüm (ZigZag ve Inverse Zigzag) Algoritmaları

Huffman encoder algoritması girdi olarak satır matrisi almaktadır. Bu kapsamda 8x8'lik matris bloklarını 1x64 formatına getirecek bir matris dönüşümüne ihtiyaç duyulmuştur. Bu ihtiyacın ZigZag algoritması ile giderilmesi hedeflenmiştir. Zigzag algoritmasının görsel açıklaması Şekil 1.2'de gösterilmiştir.



Şekil 1.2: 8x8 Matris Bloğu ZigZag Uygulaması [1]

Şekil 1.2'deki matrisin sol üstteki ilk elemanı (1,1) olarak alınsın. Matrisin sütunlarının tek sayı olduğu durumları inceleyecek olursak satır sayısı 1 veya 8 iken ZigZag algoritması sağa gitme yani satır numarasını sabit tutmak koşulu ile sütun numarasını 1 artırma eğilimindedir. Algoritmanın bu yönelimini A koşulu olarak adlandıralım. Matrisin satırlarının çift sayı olduğu durumları inceleyecek olursak sütun sayısı 1 veya 8 iken ZigZag algoritması aşağıya gitme yani sütun numarasını sabit tutmak koşulu ile satır numarasını 1 artırma eğilimindedir. Algoritmanın bu yönelimini B koşulu olarak adlandıralım. Matrisin kenar koşullarını incelediğimize göre ara koşullarını yani sağ yukarı ve sol yukarı doğrultularında gitme eğilimlerini inceleyelim. Bu kapsamda matrisin satır ve sütun numaralarının toplamının çift sayı olması durumunda ZigZag algoritması sağ üst doğrultuda ilerleme, yani satır numarasını bir azaltırken sütun numarasını bir artırma eğilimindedir. Algoritmanın bu yönelimini C koşulu olarak adlandıralım. Matrisin satır ve sütun numaralarının toplamının tek sayı olması durumunda ZigZag algoritması sol alt doğrultuda ilerleme, yani satır numarasını bir artırırken sütun numarasını bir azaltma eğilimindedir. Algoritmanın bu yönelimini D koşulu olarak adlandıralım. Tüm bunlara ek olarak Algoritmanın bir noktada durması gerekmektedir. Bu son koşul da satır numarası ve sütun numarası 8 değerine eşitken gerçekleşir. Dolayısıyla satır numarası ve sütun numarasının 8 değerine eşit olması durumunu da E koşulu olarak adlandıralım. Tüm bu koşulların kontrolü için ise değişim bayrağı tutulmalıdır. Bu ZigZag algoritmasının bütüncül olarak incelenecek olursa, ilk aşamada fonksiyona girdi olarak verilen matrisin satır sayısı girdi_satir, sütun sayısı girdi_sutun olarak tespit edilir ve $1 \times (\text{girdi_satir} * \text{girdi_sutun})$ biçiminde bir satır matrisi oluşturulur. 1'den $(\text{girdi_satir} * \text{girdi_sutun})$ değerine kadar sayan bir döngü yardımıyla $(\text{girdi_satir} * \text{girdi_sutun})$ şeklindeki matris taranır. Yapılan taramada ilk olarak $(\text{girdi_satir} * \text{girdi_sutun})$ biçimindeki matrisinin güncel elemanı (başlangıç değeri (1,1) olmak üzere her koşul

içerisinde güncellenir), $1 \times (\text{girdi_satir} * \text{girdi_sutun})$ biçimindeki matrisin güncel elemanına (döngünün o andaki değeri) kaydedilir. Kaydetme işleminin ardından yukarıda bahsedilmiş olan A koşulu gözlenir ve bu koşul sağlandığı takdirde değişim bayrağı 1 değerine çekilir. A koşulunun ardından sırasıyla E koşulu gözlemlenir, E koşulu sağlanmamışsa (if-else if yapısı) B koşulu gözlemlenir. B veya E koşulu da sağlandığı takdirde değişim bayrağı 1 değerine çekilir. A koşulu ile B, E koşulları arasında bir bayrak kontrolü yapılmamaktadır. Bunun sebebi A koşulu gerçekleştikten hemen sonra B veya E koşulunun asla gerçekleşmeyecek olmasıdır. Bu üç koşul kontrol edildikten sonra değişim bayrağı hala 0 değerinde ise önce C koşulu, C koşulunun gerçekleşmemesi durumunda ise D koşulu gözlemlenir. Tüm bu incelemelerin ardından değişim bayrağı sıfırlanır ve bir sonraki döngüye geçilir. Bütün bu işlemler döngü içerisinde $(\text{girdi_satir} * \text{girdi_sutun})$ tekrarda gerçekleşmesinin ardından $1 \times (\text{girdi_satir} * \text{girdi_sutun})$ formunda bir satır matrisi elde edilir ve bu satır matrisi fonksiyonun çıkışı olarak verilir.

Inverse ZigZag algoritması ise istenen matris satır, sütun boyutlarını ve $1 \times (\text{girdi_satir} * \text{girdi_sutun})$ formundaki satır matrisini girdi olarak alır. İstenen satır boyutunu girdi_satir ve istenen sütun boyutunu girdi_sutun olarak adlandıralım ve bu istenen değerlerde bir matris oluşturulur. 1'den $(\text{girdi_satir} * \text{girdi_sutun})$ değerine kadar sayan bir döngü yardımıyla $(\text{girdi_satir} * \text{girdi_sutun})$ şeklindeki matris taranır. Yapılan taramada $1 \times (\text{girdi_satir} * \text{girdi_sutun})$ biçimindeki matrisin güncel elemanı (döngünün o andaki değeri), $(\text{girdi_satir} * \text{girdi_sutun})$ biçimindeki matrisinin güncel elemanına (başlangıç değeri (1,1) olmak üzere her koşul içerisinde güncellenir), kaydedilir. Bu aşamadan sonraki bütün koşul gözlemleri ZigZag algoritması koşul gözlemleri ile aynıdır. Bütün bu işlemler döngü içerisinde $(\text{girdi_satir} * \text{girdi_sutun})$ tekrarda gerçekleşmesinin ardından $(\text{girdi_satir} * \text{girdi_sutun})$ biçiminde bir matrisi elde edilir ve bu matris fonksiyonun çıkışı olarak verilir.

Huffman Kodlaması ve Kod Çözümü

Projenin bu kısmında ZigZag algoritmasından çıkmış olan sayı dizisi Huffman kodlama ile binary bir kod dizisine dönüştürülmektedir. Huffman kodlaması ZigZag çıktısında en çok tekrar eden değerleri en kısa kod, en az tekrar eden değerleri en uzun kod ile göstererek kayıpsız bir sıkıştırma sağlamaktadır. Algoritmada öncelikle ZigZag çıktısındaki değerler "getMsg" fonksiyonu ile tespit edilmektedir. Bu değerlerin olasılığı tekrar etme sayısı ve ZigZag çıktısının uzunluğuna göre "getProbs" fonksiyonu ile hesaplanmaktadır. Her bir değer olasılığı hesaplandıktan sonra bu değerler "getHuffman" fonksiyonuna girdi olarak verilmektedir ve çıktı olarak Huffman tablosu elde edilmektedir. Bu fonksiyonda ilk olarak tablonun birinci sütununa elde edilen sayı seti ve üçüncü sütununa bu sayıların olasılıkları yazılmaktadır. Ana tablonun yanı sıra geçici bir tablo tutulmaktadır. Bu tabloda iteratif olarak en küçük olasılıklı değerler olasılıkları toplanarak birleştirilmektedir, bu olasılık toplamalarının bire eşit olması ile döngü son bulmaktadır ve Huffman tablosunun doldurulması sonuçlanmaktadır. En küçük olasılıklar toplandıktan sonra bir sonraki

adımdaki en küçük olasılığın şu ankinin tabloda yukarısında mı aşağısında mı olmasına göre bu olasılıktaki, (sonraki) değişken değerine karşılık gelen kodun başına bir veya sıfır eklenmektedir ve bu kodlar her iterasyonda tablonun ikinci sütununa eklenmektedir. Bütün tablo doldurulduğunda tablo fonksiyon çıktısı olarak dönmektedir.

Daha sonrasında elde edilen Huffman tablosu ve ZigZag algoritmasının çıktısı “huffmanEncode” fonksiyonuna girdi olarak sağlanmaktadır. Bu fonksiyon tablo ve değişken üzerinden iterasyon yaparak her değişken değerini o değere karşılık gelen kod parçası ile eleştirmektedir. Eşleşen kodları art arda ekleyerek bütün ZigZag çıktısı değişkenine karşılık gelen kod tümcesini oluşturmaktadır.

Projenin bu kısmında “huffmanDecoder” fonksiyonu Huffman kodlamasından elde edilen kod tümcesini ve Huffman tablosunu girdi olarak almaktadır. Bu fonksiyonda, kodlamadan elde edilen kod tümcesinin her biti Huffman tablosundaki her kodun her biti ile karşılaştırılmaktadır. ZigZag çıktısı değerine karşılık gelen bir kod tespit edildiğinde kod tümcesinden bu tespit edilen kod parçası silinir ve çıktı değişkenine, bu koda karşılık gelen quantization değeri, yani ZigZag çıktısı değeri kaydedilir. Bu işlem kod tümcesi tamamen silinene kadar devam eder ve bu şekilde tüm kod tümcesinin kod çözümü yapılmış olur, kod çözümü algoritmasının çıkışında ZigZag algoritması çıkışı aynen elde edilmiş olur. Doğru kod çözümü bu şekilde tamamlanmış olur.

Inverse DCT ve Dequantization İşlemi

Projenin bu kısmında inverse ZigZag algoritmasından çıkmış olan görüntü bloklarının her biri verilmiş olan quantization tablosu ile çarpılıp dequantization işlemi yapılmaktadır. Bu işlem sonucunda quantization işlemine giren blokların her birinin her değerinin en yakın tam sayıya yuvarlanmış halleri elde edilmektedir. Bu durum kayıplı sıkıştırmanın bir sonucudur. Quantization ve dequantization sonucu tıpatıp aynı değerlerin elde edilmesi mümkün değildir. Dequantization işleminden sonra DCT işleminde yapılan matris işlemlerinin tam tersi yapılmaktadır ve her bir blok DCT matrisi ile tekrar iki kez çarpılmaktadır (DCT matrisinin dequantize edilmiş görüntü bloğu ile çarpılması ve bu elde edilen matrisin DCT matrisinin transpozu ile çarpılması). Bu işlemler sonucunda her bir görüntü bloğu dequantize edilmiş olmaktadır ve frekans alanından uzaysal alana dönmektedir ve JPEG kod çözümü bu şekilde tamamlanmış olmaktadır.

Soru-2

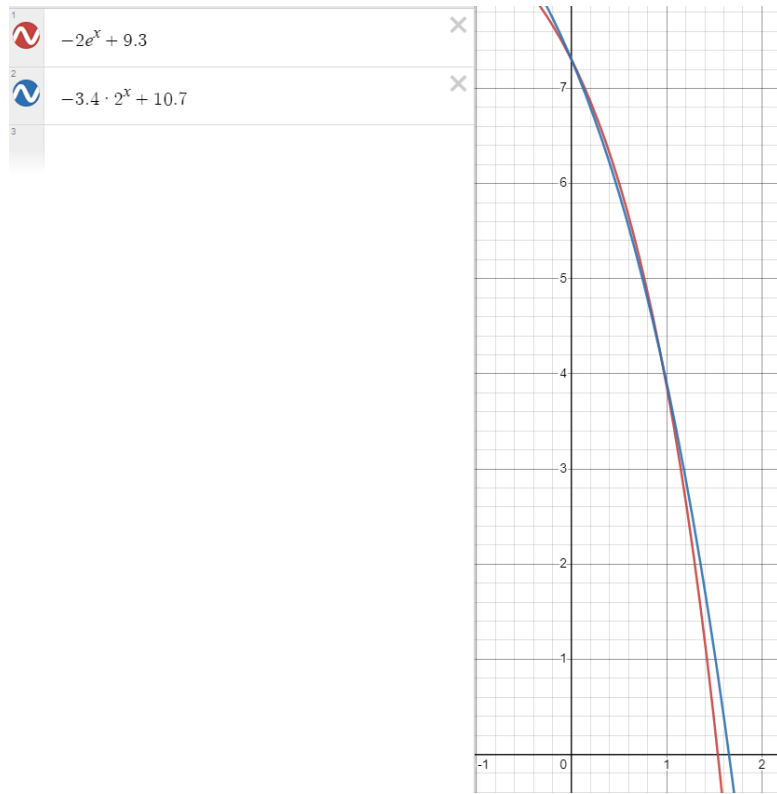
Projenin ikinci kısmı kapsamında mevcut JPEG Encoder ve JPEG Decoder algoritmalarında kullanılan Quantization matrisini bir katsayı ile çarparak birden fazla Quantization matrisi oluşturulması ve bunu sağlayan katsayıları da bir matris olarak depolanması istenmektedir. Bu kapsamda görselin piksel değerleri incelenebilir. Bir görselin piksellerindeki değişimin çok olması o piksellerin Quantization işleminin daha hassas yapılmasını gerektirir. Quantization işlemi büyük çarpanlar ile yapıldığı takdirde bu yüksek değişim noktalarını birbirlerine daha yaklaştırarak görselin keskinliğini, dolayısıyla kalitesini düşürdüğünü söyleyebiliriz. O halde pikseller arası değer değişimlerinin yüksek olduğu yerlerin fazla bilgi taşıdığını söyleyebiliriz. Bunun paralelinde görselin piksellerindeki değişimin az olması o piksellerin Quantization işleminin daha az hassas yapılmasına olanak sağlar. Quantization işlemi büyük çarpanlar ile yapıldığı takdirde bu az değişim noktalarını birbirlerine yaklaştırarak görselin tek ton olduğu bölgelerde tek renk olmasına sebep olduğunu söyleyebiliriz. Bu işlem de görselin kalitesini düşürürken keskin bir değişimi düzeltmek ile kıyaslandığında çok daha küçük bir görsel kalitesi düşüşüne sebep olduğunu söylemek mümkündür. O halde pikseller arası değer değişimlerinin düşük olduğu yerlerin az bilgi taşıdığını söyleyebiliriz. Tüm bu incelemelere ek olarak Discrete Cosine Transform ile birlikte frekans alanında incelenen görselin frekansının yüksek olduğu bölgelerin fazla bilgi taşıyan pikselleri ifade ettiğini, görselin frekansının düşük olduğu bölgelerin az bilgi taşıyan pikselleri ifade ettiğini söyleyebiliriz. Bütün bu kalite odaklı gözlemlerin paralelinde çok hassas yani çok düşük Quantization matrisleri ile işlem yapıldığı takdirde görseli sıkıştırma oranının çok düştüğünü, yüksek Quantization matrisleri ile de görseli sıkıştırma oranının çok arttığını ancak bu durumda da kalitenin çok düştüğünü gözlemlemek mümkündür. Bu kapsamda kalite parametresi olarak PSNR değerini minimum 30 olarak sabitleyip maksimum sıkıştırma oranı elde edilmeye çalışılmıştır.

Bütün bu bilgiler paralelinde ilk olarak Quantization katsayıları uygulanan metodun çarpmaya göre tersi (x/metot , x değeri PSNR 30'a sabitlemek için değiştirilen bir parametredir) şeklinde bir yöntem ile hesaplanmıştır. Bu kapsamda aşağıdaki 3 metod sırası ile test edilmiştir.

- **Pikselin Türevsel Ortalaması:** Verilen görselin piksellerinin mutlak değerlerinin türevi alınarak yeni bir 8x8 matris elde edilmiştir. Bu elde edilen 8x8 matrisin ortalamaları alınmıştır ve PSNR 30 değeri için sıkıştırma oranı incelenmiştir. Amaç piksel değişimlerinin anlık yükseldiği noktaları tespit etmektir.
- **Frekansın Türevsel Ortalaması:** Görselin DCT alındıktan sonra elde edilen frekanslarının mutlak değerlerinin türevi alınarak yeni bir 8x8 matris elde edilmiştir. Bu elde edilen 8x8 matrisin ortalamaları alınmıştır ve PSNR 30 değeri için sıkıştırma oranı incelenmiştir. Amaç yüksek bilgi ve düşük bilgi içeren frekans bloklarını tespit etmektir.

- Pikselin FFT'sinin Ortalaması: Görselin FFT alındıktan sonra elde edilen frekans değerleri ile yeni bir 8x8 matris elde edilmiştir. Bu elde edilen 8x8 matrisin ortalamaları alınmıştır ve PSNR 30 değeri için sıkıştırma oranı incelenmiştir. Amaç yüksek bilgi ve düşük bilgi içeren frekans bloklarını tespit etmektir.
- Frekansın Entropisi: Görselin DCT alındıktan sonra elde edilen frekanslarının mutlak değerlerinin entropisi alınmış ve PSNR 30 değeri için sıkıştırma oranı incelenmiştir. Amaç yüksek bilgi ve düşük bilgi içeren frekans bloklarını tespit etmektir.
- Pikselin Entropisi: Verilen görselin piksellerinin entropisi alınmış ve PSNR 30 değeri için sıkıştırma oranı incelenmiştir. Amaç piksellerin değişimlerinin yoğun olduğu bölgeleri tespit etmektir.

Bu metotların hepsi test edilmiş ve gözlemlenmiştir. Yapılan gözlemler sonucunda en iyi sonucun Piksel Entropisi alınarak elde edilen Quantization katsayıları olduğu tespit edilmiştir. İkinci aşama olarak ise Pikselin Entropisi metodu sabit tutularak Quantization katsayıları uygulanan metodun çarpmaya göre tersi (x/metot , x değeri PSNR 30'a sabitlemek için değiştirilen bir parametredir) şeklinde tanımladığımız yöntemi değiştirdik. Bu kapsamda ilk soruda hesaplanan PSNR 35 değerini 30'a kadar düşürebileceğimiz için Quantization katsayılarının 1'den büyük değerler alması gerektiği hesaplanmıştır. Bu gözlem paralelinde dene-yanıl metodu ile maksimum entropi değerinde Quantization katsayısının 4 ± 0.2 , minimum entropi değerinde Quantization katsayısının 7.2 ± 0.3 olduğu tespit edilmiştir. Bu gözlem sonucunda bu iki aralığa önce lineer, ardından ise eğri uydurarak bir hesaplama yöntemi geliştirilmiştir. Bu kapsamda sıkıştırma oranı açısından en verimli iki eğri ve fonksiyonları Şekil 2.1'de gösterilmiştir.



Şekil 2.1: En Verimli İki Eğri Denklemi ve Grafiği

Şekil 2.1’de verilen eğrilerin verimliliği Piksel Entropisi yöntemi ile test edilmiştir. Bu yöntem için exponential fonksiyonun sıkıştırma oranı daha iyi sonuç vermiştir. Hem exponential fonksiyon hem de üstel fonksiyon diğer metotlarda da test edilmiş ve test sonuçları Tablo 2.1’de verilen tabloda gösterilmiştir. Gösterilen veriler her metot için maksimum entropi değerinde Quantization katsayısının 4 ± 0.2 , minimum entropi değerinde Quantization katsayısının 7.2 ± 0.3 sınırları içerisinde yer alacak şekilde fonksiyonların düzenlenmesi ile oluşturulmuş ve tabloda verilen değerler en verimli fonksiyonun sıkıştırma oranına göre yazılmıştır.

Yöntem	PSNR	SSIM	Compression Ratio (Bitwise Image Length / Bitwise Code Length)
JPEG	30.079	0.8133	7.2865
DCT Entropy	30.0804	0.8171	7.2288
Image Entropy	30.0163	0.8086	7.2826
Image Gradient	30.0842	0.8039	6.7488
DCT Gradient	29.8831	0.8071	7.0973
Image FFT	29.8618	0.8076	7.0573

Tablo 2.1 Yöntem - PSNR - Sıkıştırma Oranı Tablosu

Sonuç

Proje kapsamında bir JPEG encoder ve JPEG decoder algoritması yazıldı. Bu algoritma yazılırken ilk aşamada Discrete Cosine Transform ve Quantization işlemleri detaylı olarak incelenerek konuya hakim olundu. Bu konuların derste teorik olarak işlenmesi sırasında dersin projesi sebebiyle pratik yapılmış olması konunun çok daha iyi kavranmasını sağladı. Bir sonraki aşama olan ZigZag algoritması ve Huffman algoritması algoritmik düşünme ve değerlendirme yetilerini geliştirirken Huffman ve benzeri sıkıştırma algoritmaları da bu süreçte incelenerek bu alanda daha da detaylı bilgi edinildi. Tüm bu işlemlerin tersleri alınarak JPEG decoder işleminin yapılması gündelik hayatta sıklıkla kullandığımız bu formatın aslında ne kadar önemli ve gerekli olduğunun kavranması konusunda bizlere yardımcı oldu. Projenin devam aşaması olan Adaptive Quantization konusu ise bu alanda araştırma ve geliştirmelerin ne kadar hassas ve tecrübe isteyen bir alan olduğunu bizlere gösterirken bunun yanında bu alana olan ilgi ve merakımızı keşfetmemizi sağladı.

Soru-1 Kodu

```
clear all
close all
clc
image = imread('lena.bmp');
A = double(image)-128;
[M,N] = size(A);
blockLen = 8;
dctMat = dctMatrixFunc(blockLen);
quantizMat = [16 11 10 16 24 40 51 61; % Default quantization table
              12 12 14 19 26 58 60 55; % compressionRatio = 5.6862
              14 13 16 24 40 57 69 56; % psnrImage = 36.5892
              14 17 22 29 51 87 80 61;
              18 22 37 56 68 109 103 77;
              24 35 55 64 81 104 113 92;
              49 64 78 87 103 121 120 101;
              72 92 95 98 112 100 103 99];

%% DCT + Quantization
numIterRow = M/blockLen;
numIterCol = N/blockLen;
imageBlockMat = zeros(blockLen,blockLen,numIterRow,numIterCol);
for i = 1:numIterRow
    for j = 1:numIterCol
        imageBlock =
A((i-1)*blockLen+1:i*blockLen, (j-1)*blockLen+1:j*blockLen);
        imageBlockMat(:,:,i,j) = round(dctMat*imageBlock*dctMat'./quantizMat);
    end
end
%% Zigzag Huffman Encoding
zigzagArr = zeros(1, blockLen*blockLen, numIterRow, numIterCol);
for i = 1:numIterRow
    for j = 1:numIterCol
        zigzagArr(:,:, i, j) = zigzagFunc(imageBlockMat(:,:,i,j));
    end
end
zigzagReshaped = reshape(zigzagArr,1,[]);
numSet = getMsg(zigzagReshaped);
probs = getProbs(zigzagReshaped, numSet);
table = getHuffman(numSet, probs);
code = huffmanEncoder(zigzagReshaped, table);
%% Huffman Decoding Inv Zigzag
decodedStr = huffmanDecoder(code, table);
decodedStr = reshape(decodedStr,64,length(decodedStr)/64);
decodedStr = decodedStr';
imageBlockMatDecoded = zeros(size(imageBlockMat));
for i = 1:numIterRow
    for j = 1:numIterCol
        index = (i-1)*64+j;
        imageBlockMatDecoded(:,:, j, i) =
inv_zigzagFunc(decodedStr(index,:),8,8);
    end
end
```

```

end
%% Dequantization + Inverse DCT
dqImageBlockMat = imageBlockMatDecoded.*quantizMat;
for i = 1:numIterRow
    for j = 1:numIterCol
        idctImageBlock = dctMat'*dqImageBlockMat(:, :, i, j)*dctMat;

idctImageBlockMat((i-1)*blockLen+1:i*blockLen, (j-1)*blockLen+1:j*blockLen) =
idctImageBlock;
    end
end
imageReconstr = uint8(idctImageBlockMat + 128);
figure
psnrImage = psnr(imageReconstr, image)
ssimImage = ssim(imageReconstr, image)
compressionRatio = (512*512*8)/length(code)
subplot(1,2,1), imshow(image)
title("Original Image")
subplot(1,2,2), imshow(imageReconstr)
title("Reconstructed Image")
imwrite(imageReconstr, 'lenaReconstructed.bmp')
%% Functions
function T = dctMatrixFunc(M)
    T = zeros(M);
    for p = 0:M-1
        for q = 0:M-1
            if p ~= 0
                T(p+1, q+1) = sqrt(2/M)*cos(pi*(2*q+1)*p/(2*M));
            else
                T(p+1, q+1) = 1/sqrt(M);
            end
        end
    end
end
function zigzagArray = zigzagFunc(matrix)
    [m, n] = size(matrix);
    zigzagArray = zeros(1, m * n);
    matrixColumn = 1;
    matrixRow = 1;
    changedBit = 0;
    for indexZigZag = 1 : m * n
        zigzagArray(indexZigZag) = matrix(matrixRow, matrixColumn);
        if mod(matrixColumn, 2) == 1
            if matrixRow == 1 || matrixRow == 8
                matrixColumn = matrixColumn + 1;
                changedBit = 1;
            end
        end
        if mod(matrixRow, 2) == 0
            if matrixRow == 8 && matrixColumn == 8
                changedBit = 1;
            elseif matrixColumn == 1 || matrixColumn == 8

```

```

        matrixRow = matrixRow + 1;
        changedBit = 1;
    end
end

if changedBit == 0
    if mod((matrixColumn + matrixRow), 2) == 0
        matrixColumn = matrixColumn + 1;
        matrixRow = matrixRow - 1;
    elseif mod((matrixColumn + matrixRow), 2) == 1
        matrixRow = matrixRow + 1;
        matrixColumn = matrixColumn - 1;
    end
end
changedBit = 0;
end
end
function matrix = inv_zigzagFunc(zigzagArray, wantedRow, wantedColumn)
    matrix = zeros(wantedRow, wantedColumn);
    matrixColumn = 1;
    matrixRow = 1;
    changedBit = 0;
    for indexZigZag = 1 : wantedRow * wantedColumn
        matrix(matrixRow, matrixColumn) = zigzagArray(indexZigZag);
        if mod(matrixColumn, 2) == 1
            if matrixRow == 1 || matrixRow == 8
                matrixColumn = matrixColumn + 1;
                changedBit = 1;
            end
        end

        if mod(matrixRow, 2) == 0
            if matrixRow == 8 && matrixColumn == 8
                changedBit = 1;
            elseif matrixColumn == 1 || matrixColumn == 8
                matrixRow = matrixRow + 1;
                changedBit = 1;
            end
        end

        if changedBit == 0
            if mod((matrixColumn + matrixRow), 2) == 0
                matrixColumn = matrixColumn + 1;
                matrixRow = matrixRow - 1;
            elseif mod((matrixColumn + matrixRow), 2) == 1
                matrixRow = matrixRow + 1;
                matrixColumn = matrixColumn - 1;
            end
        end
        changedBit = 0;
    end
end
function out = huffmanDecoder(code, table)

```

```

tmp = [];
out = [];
k = 1;
while length(code) ~= 0
    for i = 1:length([table{:,1}])
        if isequal(tmp, table{i,2})
            out = [out, table{i,1}];
            code(1:length(tmp)) = [];
            tmp = [];
            k = 1;
        end
    end
    if ~isempty(code)
        tmp = [tmp, code(k)];
    end
    k = k + 1;
end
end
function codeword = huffmanEncoder(str,table)
codeword = [];
for i = 1:length(str)
    index = strfind([table{:,1}], str(i));
    codeword = [codeword, table{index,2}];
end
end
function table = getHuffman(msgSet, probs)
[sortedProbs, sort_ind] = sort(probs, 'descend');
sortedMsgSet = msgSet(sort_ind);
for i = 1:length(sortedMsgSet)
    table{i,1} = sortedMsgSet(i);
    table{i,2} = '';
    table{i,3} = sortedProbs(i);
end
auxTable = table;
flag = true;
while flag
    for j = 1:length(auxTable{end-1,1})
        upInd = strfind([table{:,1}], auxTable{end-1,1}(j));
        table{upInd,2} = ['1', table{upInd,2}];
    end
    for j = 1:length(auxTable{end,1})
        downInd = strfind([table{:,1}], auxTable{end,1}(j));
        table{downInd,2} = ['0', table{downInd,2}];
    end
    auxTable{end-1,1} = [auxTable{end-1,1}, auxTable{end,1}];
    auxTable{end-1,3} = auxTable{end-1,3} + auxTable{end,3};
    auxTable(end,:) = [];
    [~, sort_ind] = sort([auxTable{:,3}], 'descend');
    for i = 1:length(sort_ind)
        tmp{i,1} = auxTable{sort_ind(i),1};
        tmp{i,2} = auxTable{sort_ind(i),2};
        tmp{i,3} = auxTable{sort_ind(i),3};
    end
end

```



```

    auxTable = tmp;
    tmp = [];
    if ((auxTable{end,3} + auxTable{end-1,3}) == 1)
        flag = false;
    end
end
for j = 1:length(auxTable{end-1,1})
    upInd = strfind([table{:,1}], auxTable{end-1,1}(j));
    table{upInd,2} = ['1', table{upInd,2}];
end
for j = 1:length(auxTable{end,1})
    downInd = strfind([table{:,1}], auxTable{end,1}(j));
    table{downInd,2} = ['0', table{downInd,2}];
end
end
function probs = getProbs(str, msgSet)
    strLength = length(str);
    setSize = length(msgSet);
    for idx = 1:setSize
        probs(idx) = sum(str == msgSet(idx))/strLength;
    end
end
function msgSet = getMsg(str)
    strLength = length(str);
    msgSet = [];
    for i = 1:strLength
        if isempty(find(msgSet == str(i)))
            msgSet = [msgSet, str(i)];
        end
    end
end
end

```

Soru-2 Kodu

```
clear all
close all
%clc
image = imread('lena.bmp');
A = double(image)-128;
[M,N] = size(A);
blockLen = 8;
dctMat = dctMatrixFunc(blockLen);
quantizMat = [16 11 10 16 24 40 51 61; % Default quantization table
% Adaptive: %30 PSNR'da sabitken
12 12 14 19 26 58 60 55; % compressionRatio = 7.2865
% compressionRatio = 7.2826
14 13 16 24 40 57 69 56; % psnrImage = 30.079
% psnrImage = 30.0163
14 17 22 29 51 87 80 61; % ssimImage = 0.8133
% ssimImage = 0.8086
18 22 37 56 68 109 103 77;
24 35 55 64 81 104 113 92;
49 64 78 87 103 121 120 101;
72 92 95 98 112 100 103 99];

%% DCT + Quantization
numIterRow = M/blockLen;
numIterCol = N/blockLen;
imageBlockMat = zeros(blockLen,blockLen,numIterRow,numIterCol);
for i = 1:numIterRow
    for j = 1:numIterCol
        imageBlock =
A((i-1)*blockLen+1:i*blockLen, (j-1)*blockLen+1:j*blockLen);
        dctBlock = dctMat*imageBlock*dctMat';
        entropyImBlock = entropy(imageBlock);
        entropyImBlockMat(i,j) = entropyImBlock;
        AdapQParams(i,j) = -2*exp(entropyImBlockMat(i,j)) + 9.3;
        imageBlockMat(:, :, i, j) =
round(dctBlock./ (AdapQParams(i,j)*quantizMat));
    end
end
%% Zigzag Huffman Encoding
zigzagArr = zeros(1, blockLen*blockLen, numIterRow, numIterCol);
for i = 1:numIterRow
    for j = 1:numIterCol
        zigzagArr(:, :, i, j) = zigzagFunc(imageBlockMat(:, :, i, j));
    end
end
zigzagReshaped = reshape(zigzagArr,1,[]);
numSet = getMsg(zigzagReshaped);
probs = getProbs(zigzagReshaped, numSet);
table = getHuffman(numSet, probs);
code = huffmanEncoder(zigzagReshaped, table);
%% Huffman Decoding Inv Zigzag
decodedStr = huffmanDecoder(code, table);
```

```

decodedStr = reshape(decodedStr,64,length(decodedStr)/64);
decodedStr = decodedStr';
imageBlockMatDecoded = zeros(size(imageBlockMat));
for i = 1:numIterRow
    for j = 1:numIterCol
        index = (i-1)*64+j;
        imageBlockMatDecoded(:, :, j, i) =
inv_zigzagFunc(decodedStr(index,:),8,8);
    end
end
%% Dequantization + Inverse DCT
dqImageBlockMat = imageBlockMatDecoded.*quantizMat;
for i = 1:numIterRow
    for j = 1:numIterCol
        idctImageBlock =
dctMat'*(dqImageBlockMat(:, :, i, j)*(AdapQParams(i, j)))*dctMat;

idctImageBlockMat((i-1)*blockLen+1:i*blockLen, (j-1)*blockLen+1:j*blockLen) =
idctImageBlock;
    end
end
imageReconst = uint8(idctImageBlockMat + 128);
figure
psnrImage = psnr(imageReconst,image)
ssimImage = ssim(imageReconst,image)
compressionRatio = (512*512*8)/length(code)
imageJPEG = imread('lenaJPEG.bmp');
subplot(1,3,1),imshow(image)
title("Original Image")
subplot(1,3,2),imshow(imageJPEG)
title("JPEG Image")
subplot(1,3,3),imshow(imageReconst)
title("Reconstructed Image with adaptiveQ")
imwrite(imageReconst,'lenaReconstructed_AdaptiveQ.bmp')
%% Functions
function T = dctMatrixFunc(M)
    T = zeros(M);
    for p = 0:M-1
        for q = 0:M-1
            if p ~= 0
                T(p+1, q+1) = sqrt(2/M)*cos(pi*(2*q+1)*p/(2*M));
            else
                T(p+1, q+1) = 1/sqrt(M);
            end
        end
    end
end
function zigzagArray = zigzagFunc(matrix)
    [m, n] = size(matrix);
    zigzagArray = zeros(1, m * n);
    matrixColumn = 1;
    matrixRow = 1;
    changedBit = 0;

```

```

for indexZigZag = 1 : m * n
    zigzagArray(indexZigZag) = matrix(matrixRow, matrixColumn);
    if mod(matrixColumn, 2) == 1
        if matrixRow == 1 || matrixRow == 8
            matrixColumn = matrixColumn + 1;
            changedBit = 1;
        end
    end

    if mod(matrixRow, 2) == 0
        if matrixRow == 8 && matrixColumn == 8
            changedBit = 1;
        elseif matrixColumn == 1 || matrixColumn == 8
            matrixRow = matrixRow + 1;
            changedBit = 1;
        end
    end

    if changedBit == 0
        if mod((matrixColumn + matrixRow), 2) == 0
            matrixColumn = matrixColumn + 1;
            matrixRow = matrixRow - 1;
        elseif mod((matrixColumn + matrixRow), 2) == 1
            matrixRow = matrixRow + 1;
            matrixColumn = matrixColumn - 1;
        end
    end
    changedBit = 0;
end
end

function matrix = inv_zigzagFunc(zigzagArray, wantedRow, wantedColumn)
    matrix = zeros(wantedRow, wantedColumn);
    matrixColumn = 1;
    matrixRow = 1;
    changedBit = 0;
    for indexZigZag = 1 : wantedRow * wantedColumn
        matrix(matrixRow, matrixColumn) = zigzagArray(indexZigZag);
        if mod(matrixColumn, 2) == 1
            if matrixRow == 1 || matrixRow == 8
                matrixColumn = matrixColumn + 1;
                changedBit = 1;
            end
        end

        if mod(matrixRow, 2) == 0
            if matrixRow == 8 && matrixColumn == 8
                changedBit = 1;
            elseif matrixColumn == 1 || matrixColumn == 8
                matrixRow = matrixRow + 1;
                changedBit = 1;
            end
        end
    end
end

```

```

        if changedBit == 0
            if mod((matrixColumn + matrixRow), 2) == 0
                matrixColumn = matrixColumn + 1;
                matrixRow = matrixRow - 1;
            elseif mod((matrixColumn + matrixRow), 2) == 1
                matrixRow = matrixRow + 1;
                matrixColumn = matrixColumn - 1;
            end
        end
        changedBit = 0;
    end
end
function out = huffmanDecoder(code, table)
    tmp = [];
    out = [];
    k = 1;
    while length(code) ~= 0
        for i = 1:length([table(:,1)])
            if isequal(tmp, table{i,2})
                out = [out, table{i,1}];
                code(1:length(tmp)) = [];
                tmp = [];
                k = 1;
            end
        end
        if ~isempty(code)
            tmp = [tmp, code(k)];
        end
        k = k + 1;
    end
end
function codeword = huffmanEncoder(str, table)
    codeword = [];
    for i = 1:length(str)
        index = strfind([table(:,1)], str(i));
        codeword = [codeword, table{index,2}];
    end
end
function table = getHuffman(msgSet, probs)
    [sortedProbs, sort_ind] = sort(probs, 'descend');
    sortedMsgSet = msgSet(sort_ind);
    for i = 1:length(sortedMsgSet)
        table{i,1} = sortedMsgSet(i);
        table{i,2} = '';
        table{i,3} = sortedProbs(i);
    end
    auxTable = table;
    flag = true;
    while flag
        for j = 1:length(auxTable{end-1,1})
            upInd = strfind([table(:,1)], auxTable{end-1,1}(j));
            table{upInd,2} = ['1', table{upInd,2}];
        end
    end
end

```

```

for j = 1:length(auxTable{end,1})
    downInd = strfind([table{:,1}], auxTable{end,1}(j));
    table{downInd,2} = ['0', table{downInd,2}];
end
auxTable{end-1,1} = [auxTable{end-1,1}, auxTable{end,1}];
auxTable{end-1,3} = auxTable{end-1,3} + auxTable{end,3};
auxTable(end,:) = [];
[~, sort_ind] = sort([auxTable{:,3}], 'descend');
for i = 1:length(sort_ind)
    tmp{i,1} = auxTable{sort_ind(i),1};
    tmp{i,2} = auxTable{sort_ind(i),2};
    tmp{i,3} = auxTable{sort_ind(i),3};
end
auxTable = tmp;
tmp = [];
if ((auxTable{end,3} + auxTable{end-1,3}) == 1)
    flag = false;
end
end
for j = 1:length(auxTable{end-1,1})
    upInd = strfind([table{:,1}], auxTable{end-1,1}(j));
    table{upInd,2} = ['1', table{upInd,2}];
end
for j = 1:length(auxTable{end,1})
    downInd = strfind([table{:,1}], auxTable{end,1}(j));
    table{downInd,2} = ['0', table{downInd,2}];
end
end
function probs = getProbs(str, msgSet)
    strLength = length(str);
    setSize = length(msgSet);
    for idx = 1:setSize
        probs(idx) = sum(str == msgSet(idx))/strLength;
    end
end
function msgSet = getMsg(str)
    strLength = length(str);
    msgSet = [];
    for i = 1:strLength
        if isempty(find(msgSet == str(i)))
            msgSet = [msgSet, str(i)];
        end
    end
end
end

```

Kaynakça

[1] ZigZag Algoritması: en.wikipedia.org/wiki/JPEG#/media/File:JPEG_ZigZag.svg